

Finetuning Report — First Iteration

Model: CodeLlama-7B-Instruct-HF · **Method:** QLoRA (4-bit) · **Platform:** Kaggle T4 GPU
Date: July 25, 2026 · **Status:** Completed Successfully (after 4 attempts)

1. Overview

This report documents the first finetuning experiment for the **ITS AI** project, where `codellama/CodeLlama-7b-Instruct-hf` was fine-tuned using the **QLoRA** technique (Quantized Low-Rank Adaptation) on a custom Indonesian student code assessment dataset. The goal is to teach the model to evaluate student Python code and generate structured scoring + feedback in Bahasa Indonesia.

2. Training Environment

Property	Value
Platform	Kaggle Notebook
GPU	2× NVIDIA Tesla T4 (15,360 MiB each)
CUDA Version	13.0
Driver Version	580.159.04
Python Version	3.12.13
Framework	PyTorch + HuggingFace Transformers
PEFT Version	0.19.1
TRL	SFTTrainer / SFTConfig
Quantization	BitsAndBytes 4-bit (NF4)

3. Dataset

Sources

Three datasets were used from Kaggle (`hanzfr` account):

- `/kaggle/input/datasets/hanzfr/soal-ujian/soal_ujian.json` — exam questions + expected outputs
- `/kaggle/input/datasets/hanzfr/nilai-ujian/nilai_ujian.json` — student scores & feedback
- `/kaggle/input/datasets/hanzfr/train-data/train_data.jsonl` — pre-built training data

Dataset Statistics

Metric	Value
Total Usable Examples	1,512
Min Token Length	274 tokens
Max Token Length	1,151 tokens
Mean Token Length	~511 tokens
95th Percentile (p95)	842 tokens
Max Sequence Length (training)	1,024 tokens

Train / Validation Split

Split was performed at the **question level** (`id_soal`) to prevent data leakage — ensuring the model is evaluated on questions it has never seen, not just unseen student answers for the same question.

Split	Count	Proportion
Train	1,296 examples	~85.7%
Validation	216 examples	~14.3%

Prompt Format

Examples were formatted using CodeLlama's instruction template:

```
<s>[INST] Soal: {exam_question}
Output yang diharapkan: {expected_output}
```

Kode siswa:

```
{student_code}
```

Nilai kode siswa ini dan berikan feedback. [/INST] Penilaian:

- {criterion}: {score} ...

Rata-rata: {avg_score}

Feedback: {feedback_text}

4. Model Configuration

Base Model

- **Model:** `codellama/CodeLlama-7b-Instruct-hf`
- **Commit:** `22cb240e0292b0b5ab4c17ccd97aa3a2f799cbcd`
- **Task Type:** Causal Language Modeling (CAUSAL_LM)

Quantization (QLoRA)

Setting	Value
Quantization	4-bit (load_in_4bit)
Quantization Type	NF4 (Normal Float 4)
Compute Dtype	float16
Double Quantization	Enabled

LoRA Adapter Configuration

Parameter	Value
Rank (r)	16
LoRA Alpha	32
LoRA Dropout	0.1
Bias	none
Use DoRA	false
Use RSLoRA	false
Target Modules	<code>q_proj</code> , <code>k_proj</code> , <code>v_proj</code> , <code>o_proj</code>

The adapter targets all four attention projection matrices (query, key, value, output), which is a comprehensive but memory-efficient configuration for QLoRA.

Training Hyperparameters

Hyperparameter	Value
Epochs	3
Per-device Batch Size	2
Gradient Accumulation Steps	8
Effective Batch Size	16
Learning Rate	2e-4
LR Scheduler	Cosine
Warmup Ratio	0.03
Optimizer	paged_adamw_8bit
Precision	fp16
Max Sequence Length	1,024 tokens
Gradient Checkpointing	Enabled
Total Training Steps	243
Eval & Save Strategy	Per epoch
Logging Steps	Every 10 steps

5. Training Results

Loss Curve (Training Loss per Step)

Step	Epoch	Training Loss	Grad Norm	Learning Rate
10	0.12	1.4307	0.2577	2.000e-4
20	0.25	0.9987	0.2333	1.987e-4
30	0.37	0.7818	0.1828	1.957e-4
40	0.49	0.6961	0.1991	1.910e-4
50	0.62	0.6645	0.2723	1.846e-4
60	0.74	0.6095	0.2598	1.768e-4
70	0.86	0.5424	0.2859	1.676e-4
80	0.99	0.5178	0.2593	1.571e-4

81	1.00	—	—	—
90	1.11	0.4958	0.2570	1.457e-4
100	1.23	0.4654	0.2688	1.334e-4
110	1.36	0.4639	0.2928	1.206e-4
120	1.48	0.4411	0.2981	1.073e-4
130	1.60	0.4460	0.3540	9.399e-5
140	1.73	0.4131	0.3071	8.074e-5
150	1.85	0.4233	0.2977	6.783e-5
160	1.98	0.4389	0.3247	5.550e-5
162	2.00	—	—	—
170	2.10	0.4007	0.2859	4.396e-5
180	2.22	0.3841	0.3349	3.342e-5
190	2.35	0.3904	0.3907	2.407e-5
200	2.47	0.3994	0.3614	1.607e-5
210	2.59	0.4034	0.3428	9.574e-6
220	2.72	0.3922	0.3665	4.690e-6
230	2.84	0.3919	0.3269	1.506e-6
240	2.96	0.3859	0.3729	8.041e-8
243	3.00	—	—	—

Evaluation Loss (Per Epoch)

Epoch	Eval Loss	Runtime	Samples/sec
1	0.7820	98.86s	2.185
2	0.7949	98.38s	2.196
3	0.7965	98.44s	2.194

Key Observations

- Training loss dropped dramatically from 1.4307 → 0.3859 (a ~73% reduction) across 3 epochs — a very strong signal that the model was learning the task format.

- **Gradient norms remained stable** (~0.18–0.39), indicating a healthy training run with no exploding gradients.
- **Evaluation loss plateaued early** (~0.782 at Epoch 1), then slightly increased at Epochs 2 and 3. This is a classic sign of mild **overfitting** beginning from Epoch 2 onwards.
- The **best model checkpoint** was effectively at **Epoch 1 (step 81)**, where validation loss was at its lowest (0.7820).
- Total FLOPs: $\sim 9.42 \times 10^{16}$

[!IMPORTANT] **Possible Overfitting Signal:** The training loss continued to decrease into epoch 3 (0.38), while eval loss slightly increased (0.7820 → 0.7965). This train/eval divergence suggests the model may be memorizing training examples in later epochs. Consider using the Epoch 1 checkpoint for inference.

6. Checkpoints & Saved Artifacts

Checkpoint	Step	Epoch	Adapter Size	Notes
checkpoint-81	81	1	~64 MB	Lowest eval loss (0.7820)
checkpoint-162	162	2	~64 MB	Eval loss: 0.7949
checkpoint-243	243	3	~64 MB	Eval loss: 0.7965
final_model/	243	3	~64 MB	Same as checkpoint-243

All checkpoints contain: `adapter_config.json` , `adapter_model.safetensors` , tokenizer files, `training_args.bin` , and `trainer_state.json` .

7. Troubleshooting — Session Attempts

The successful training run was achieved on the **5th attempt** after overcoming a series of library compatibility issues. Here is a summary:

Attempt	Log File	Error	Root Cause
#0	codellama-7b-instruct-qlora.log	ModuleNotFoundError: No module named 'trl'	trl package was not installed
#1	codellama-7b-instruct-qlora (1).log	TypeError: SFTConfig.__init__() got an unexpected keyword argument 'max_seq_length'	API changed in newer TRL version (max_seq_length moved)

#2	codellama-7b-instruct-qlora (2).log	AttributeError: 'functools.partial' object has no attribute '__func__'	TRL internal bug with chunked CE patching + PEFT
#3	codellama-7b-instruct-qlora (3).log	RuntimeError: Expected all tensors to be on the same device (cuda:1 and cuda:0)	Multi-GPU device_map="auto" conflict during loss computation
#4	codellama-7b-instruct-qlora (4).log	RuntimeError: chunk expects at least a 1-dimensional tensor	DataParallel scatter issue with multi-GPU setup
** Final**	(successful run)	Training completed	Fixed by constraining to single GPU or adjusting device map

8. Output Files Summary

```
finetuning_results/first_iteration/
├── __huggingface_repos__.json      # HF model download manifest
├── train_data.jsonl               # Full formatted dataset (1,512 examples, ~1.9 MB)
├── train_split.jsonl             # Training split (1,296 examples, ~1.7 MB)
├── val_split.jsonl               # Validation split (216 examples, ~238 KB)
├── checkpoints/
│   ├── checkpoint-81/            # End of Epoch 1 [Best eval loss]
│   ├── checkpoint-162/          # End of Epoch 2
│   └── checkpoint-243/          # End of Epoch 3 (= final_model)
├── final_model/
│   ├── adapter_config.json       # LoRA config
│   ├── adapter_model.safetensors # Trained LoRA weights (~64 MB)
│   ├── tokenizer.json            # Full tokenizer (~3.5 MB)
│   ├── tokenizer.model           # SentencePiece model (~488 KB)
│   └── training_args.bin         # Serialized TrainingArguments
└── logs/
    ├── codellama-7b-instruct-qlora.ipynb    # Source notebook
    ├── codellama-7b-instruct-qlora.log       # Attempt 0: trl missing
    ├── codellama-7b-instruct-qlora (1).log   # Attempt 1: max_seq_length error
    ├── codellama-7b-instruct-qlora (2).log   # Attempt 2: functools.partial error
    ├── codellama-7b-instruct-qlora (3).log   # Attempt 3: multi-GPU device mismatch
    └── codellama-7b-instruct-qlora (4).log   # Attempt 4: DataParallel scatter error
```

9. Analysis & Recommendations

What Went Well

- **Training converged cleanly** — loss reduced from 1.43 → 0.39 with stable gradients.
- **Dataset pipeline is solid** — question-level splitting prevents leakage, and the prompt format aligns with CodeLlama's expected instruction template.
- **Adapter size is lean** — ~64 MB LoRA weights on a 7B base model is highly deployment-efficient.

Concerns

1. **Overfitting risk:** Eval loss plateaued at Epoch 1 while training loss kept falling. The `checkpoint-81` (Epoch 1) should be preferred for inference over `final_model`.
2. **Eval loss gap:** The eval loss (0.78) is significantly higher than the final training loss (0.39). This gap (~0.40) is relatively large and warrants further investigation — it may indicate the model is memorizing training patterns rather than generalizing.
3. **padding_side warning:** All successful runs raised `UserWarning: padding_side not equal to 'right'`. This should be fixed by adding `tokenizer.padding_side = 'right'` before training, which can affect half-precision training stability.

Recommendations for Next Iteration

#	Recommendation	Priority
1	Use <code>checkpoint-81</code> for inference and evaluation — it had the best validation loss	High
2	Fix tokenizer padding side: add <code>tokenizer.padding_side = 'right'</code>	Medium
3	Add early stopping based on validation loss to avoid wasted compute in future runs	Medium
4	Reduce epochs to 1–2 or use a lower learning rate to combat overfitting	Medium
5	Expand validation set — 216 examples is small; try 20% split	Low
6	Qualitative evaluation — generate predictions on held-out examples and manually review	High
7	Consider constraining to single GPU in all future Kaggle runs to avoid DataParallel issues	Medium